

Lab 3 (Exercises)

Methods in Artificial Intelligence (INFO180)

In Lab 3, you will begin by formulating constraints for a Constraint Satisfaction Problem (CSP). This will help you gain a good knowledge of how to formulate such problems as logical constraints. After that, there are two additional questions to discuss the answers.

Task 1.

Download and unzip the **Lab3_Files.zip** from the *Lab 3* folder at *Mitt.uib*. There are 3 Python files in there. In the program, you find the code to select a number of guests (e.g., Anne, Ola, Jan) who can be invited to a party where constraints have been set as to what kind of guests fit together. As the program is, basically, you have three potential guests and three constraints (you can find this in the file **invitation_node.py**). You will make the data more complex by formulating constraints for 5, 10, and more potential guests.

Try different combinations by running the file **invitation_solver.py** with different constraints and a potential number of guests. What effect do more guests and more constraints have on the running time of the program? Discuss what is the cause of the differences.

To make this happen, you have to:

- a) Define a group of individuals in the constructor of the *InvitationNode* class (following the pattern used before), with names.
- b) Put them in the list *self.assignment* in the same constructor.
- c) Formulate the constraints in the same way as is done in the Boolean method *is_consistent()*. Each constraint involves some potential guests. It must be checked whether these guests have received the status of 'invited' or 'not invited' before the constraint can be checked. Therefore, there is a function called *none_undecided()* which is used there, together with the constraint itself. Such hard-coding of constraints is perhaps not so smart normally, but probably easiest to achieve in such a small program as this.
- d) In addition, the program uses a constraint that checks whether the number of guests is within an interval that you can define in the code. This is implemented with the method *is_ok_guest_count()*. Try different limits for what counts as an acceptable number of guests.

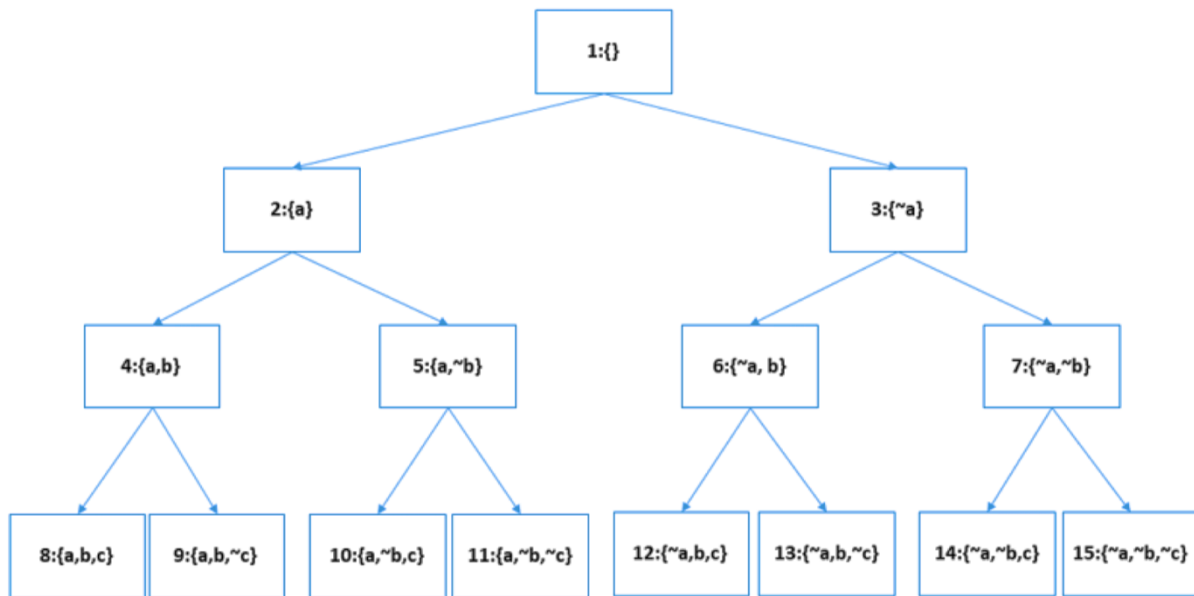
Task 2.

Check the following question and discuss your answer:

A Constraint Satisfaction Problem (CSP) can be solved by searching in a search space with partial assignments. Suppose you have the following logical constraints:

- $\sim a \wedge b$
- $b \vee \sim c$
- $\sim a \vee c$

where $\sim$ is negation. At which nodes in the search tree below will the search stop due to inconsistency?



Select one alternative:

(i) 2, 6

(ii) 3, 5, 8

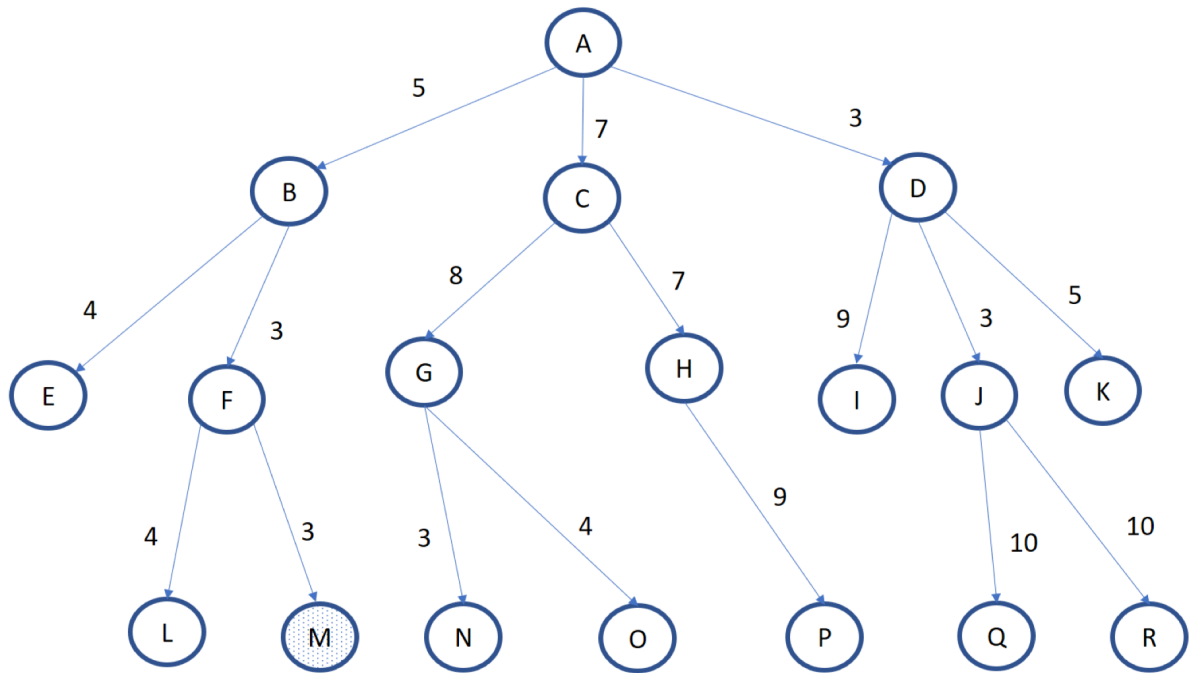
(iii) 2, 7

(iv) 4, 6, 7

Task 3.

Check the following question and discuss your answer.

The generic algorithm for searching in the state space has been provided in Lecture 2. In a **Lowest-cost-first** implementation of this algorithm, a search frontier is sorted by the length of the shortest path. The figure shows a state space organized as a tree with the node 'A' as the start node and the **node 'M'** as the target node.



How many nodes (including the target node) in this tree are visited when searching with a Lowest-cost-first organization of the search **frontier** to find the solution?

Select one alternative:

(a) 10

(b) 4

(c) 7

(d) 9